

UnionLabs SDR Platform — How to Add Your Algorithm

Contents

How to add your algorithm	1
Step 1 — Where to put it	1
One file, or many? — many.	1
Step 2 — What app.py must provide	2
Step 3 — Two ways to write app.py	2
3A. Simplest — write the algorithm inline (copy experiments/_template/)	2
3B. Link your OWN existing algorithm (copy experiments/plain_echo/)	3
Step 4 — tx vs rx (only if your two ends differ)	3
The four node types	4
Naming your own roles (optional)	4
Knowing which node you are (optional)	4
Step 5 — Run it	4
Checklist / common mistakes	5

How to add your algorithm

Follow these four steps to run your own algorithm over the SDR PHY. You write **one small file** (app.py); the framework handles the radio.

Step 1 — Where to put it

Create a folder under experiments/ whose **name is what you'll pass to run.sh**, and put an app.py inside it:

```
unionlabs/
└─ experiments/
    └─ my_algo/          <- the folder name = the algorithm name
        └─ app.py        <- REQUIRED. the framework looks for exactly this file
```

Then you run it with that same name:

```
./run.sh --algo my_algo
```

Rule: --algo <name> must match the folder name, and the folder must contain app.py. run.sh lives at the repo root; run it from there. ./run.sh list shows all algorithms found.

One file, or many? — many.

app.py is the only file the framework looks for, but it is **just the bridge**. Everything else you upload sits beside it and is imported normally — sibling modules, sub-packages, data files, model weights:

```

experiments/my_algo/
├─ app.py           <- the ONLY required file: ~10 lines of binding
├─ model.py         <- YOUR code, unchanged
├─ train_utils.py   <- more of YOUR code
├─ mypkg/           <- a sub-package works too
│   └─ __init__.py
│       └─ helper.py
└─ weights.npz      <- data / checkpoints

```

```

# app.py
from model import Model          # sibling file
from mypkg.helper import scale  # sub-package

```

The loader puts your algorithm's folder on `sys.path` before importing `app.py`, so these plain imports just work — **no `sys.path` juggling needed**. (The explicit `sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))` shown in 3B below is only needed if you also want to run `app.py` directly with `python3`, outside `run.sh`.)

Your uploaded code needs **no import from this framework and no radio code**. Only `app.py` knows the framework exists, and all it does is point transmit/receive at your methods.

Step 2 — What `app.py` must provide

`app.py` must define a function `make(role)` that returns an object with two methods:

```

def make(role):          # role is "tx" or "rx" (the framework sets it)
    return YourObject(role)

```

The returned object exposes:

Method / field	Required?	What it is
<code>transmit()</code>	yes	the numpy array to send; None when nothing is left (ends the run).
<code>receive(msg)</code>	yes	called with the received numpy array — feed it into your algorithm.
<code>spec = (dtype, shape)</code>	optional	declares your output, e.g. <code>("float32", (16,))</code> .
<code>on_result(ack)</code>	optional	called with <code>True/False</code> = delivered/lost (a reinforcement-learning reward).

That's the whole contract. No radio code, no imports from the framework.

Step 3 — Two ways to write `app.py`

3A. Simplest — write the algorithm inline (copy `experiments/_template/`)

```

# experiments/my_algo/app.py
import numpy as np

class MyAlgo:
    spec = ("float32", (8,))
    def __init__(self, role):
        self.role = role
    def transmit(self):
        return np.ones(8, np.float32)          # what to transmit (None = done)
    def receive(self, msg):
        print("got", msg)                      # what to receive

```

```
def make(role):
    return MyAlgo(role)
```

3B. Link your OWN existing algorithm (copy experiments/plain_echo/)

Leave your algorithm **untouched** in its own file next to app.py, and let app.py just map its methods. Nothing in your algorithm needs to change or import our framework.

experiments/my_algo/

```
|— my_model.py      <- YOUR existing code (unchanged)
|— app.py           <- the 10-line binding
```

my_model.py — YOUR algorithm. Knows nothing about radios.

```
import numpy as np
```

```
class MyModel:
```

```
    def __init__(self):
```

```
        self.buf = [np.array([i, i+1, i+2], np.float32) for i in range(5)]
```

```
        self.k = 0
```

```
    def next_output(self):
```

your method that produces data

```
        if self.k >= len(self.buf): return None
```

```
        out = self.buf[self.k]; self.k += 1; return out
```

```
    def take_input(self, x):
```

your method that consumes data

```
        print(" my_model received:", x)
```

app.py — the binding: map YOUR methods onto transmit()/receive()

```
import os, sys
```

```
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__))) # import files in this folder
```

```
from my_model import MyModel
```

```
def make(role):
```

```
    model = MyModel()
```

```
    class Bind:
```

```
        spec = ("float32", (3,))
```

```
        def __init__(self): self.role = role
```

```
        def transmit(self):
```

```
            return model.next_output() # <- your algorithm's output goes on the air
```

```
        def receive(self, msg):
```

```
            model.take_input(msg) # <- the received array goes into your algorithm
```

```
    return Bind()
```

Point transmit/receive at whatever your algorithm's real method names are — that's the "link". If your algorithm needs extra packages (numpy, torch, ...), just `pip install` them; the binding imports your code as-is.

Step 4 — tx vs rx (only if your two ends differ)

Each node is built with a role: **tx** (transmitter, starts by sending) or **rx** (receiver, starts by receiving and may reply). If your two ends do the *same* thing, ignore role. If they differ, branch on it — e.g. the receiver classifies and replies:

```
def transmit(self):
```

```
    if self.role == "rx":
```

```
        return np.array([self.last_answer], np.float32) # rx's reply
```

```
    return self.next_request()
```

tx's request

```
def receive(self, msg):
```

```
    if self.role == "rx":
```

```

        self.last_answer = my_model.process(msg)          # rx handles the request
    else:
        my_model.apply(msg)                               # tx handles the reply

```

(See experiments/clip_semcom/ — the tx sends an image embedding, the rx classifies it and replies the label.)

The four node types

tx/rx are two of four. A node can also be one that does **both**:

Node type	What it does	Typical use
tx	transmits first, then listens	the client / initiator
rx	listens first, then answers	the server / responder
relay	receives from upstream and re-transmits downstream	a middle hop, when the two ends are out of range
peer	both, at different steps — one node of a decentralized network	gossip / consensus, with --node K

Naming your own roles (optional)

If your algorithm calls its ends something other than tx/rx, declare the map at module level and the experimenter types **your** names:

```
ROLES = {"client": "tx", "server": "rx", "relay": "relay"}
```

```
./run.sh --algo my_algo --role server          # identical to --role rx
```

make(role) receives "server", so what is typed and what your code sees always match. tx/rx stay valid for every algorithm, and an algorithm that declares no ROLES behaves exactly as before. ./run.sh list prints each algorithm's roles.

Worked examples: experiments/fl/ (client/server/relay), experiments/dl/ (peer/initiator/responder).

Knowing which node you are (optional)

The runners that build many nodes can tell each one its position — useful when every node needs its own data shard. Widen the binding and the framework fills it in:

```
def make(role, index=None, total=None):          # index = which node, total = how many
    return MyAlgo(role, index or 0, total or 1)
```

Plain make(role) keeps working. See experiments/dl/app.py, where peer index takes shard index of total.

Step 5 — Run it

Your algorithm does not change between any of these. Only the flags do.

1) radio-free, lossless — check your logic first:

```
./run.sh --algo my_algo
```

2) through the REAL C++ modem + noise:

```
./run.sh --algo my_algo --channel usrp --sim-snr-db 6
```

3) over the LoRa PHY (no hardware needed — the sim backend is the default):

```
./run.sh --algo my_algo --channel lora --lora-sf 9
```

4) over real radios, two hosts — start the rx FIRST, then the tx:

```
./run.sh --algo my_algo --role rx --radio addr=192.168.20.2
```

```
./run.sh --algo my_algo --role tx --radio serial=30CD424 --ack-host <RX_IP>
```

5) as a multi-node network in ONE process (developing), then one node per terminal (deploying):

```
./run.sh --algo my_algo --role gossip --agents 6 --topology ring
```

```
./run.sh --algo my_algo --node 0 --agents 6
```

Add `--steps N` to control how many rounds run. `./run.sh --help` lists every option, grouped by what it configures; `BEGINNER_GUIDE.md §3.4` explains each one.

Checklist / common mistakes

- ☐ Folder is `experiments/<name>/` and `--algo <name>` matches it exactly.
- ☐ `app.py` exists and defines `make(role)`.
- ☐ `transmit()` returns a **numpy array**, or `None` to stop.
- ☐ Build state inside `make()` (per node) — **not** as module-level globals, or the two loopback ends would share state.
- ☐ Importing your own file? add `sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))` at the top of `app.py` (as in 3B).
- ☐ `spec` is optional — the wire format is self-describing, so shapes/dtypes are carried for you.
- ☐ Declared ROLES? Every value must be one of `tx`, `rx`, `relay`, `peer`.
- ☐ Your algorithm should contain **no PHY knobs**. If it needs a spreading factor or a modulation scheme, it has stopped being portable — those belong on the command line.

More detail: the full contract and the framework functions are in `experiments/README.md`; every `run.sh` option is explained in `BEGINNER_GUIDE.md §3.4`; the USRP PHY's own interface is in `drivers/usrp/GUIDE.md` and the LoRa PHY's in `drivers/lora/README.md`.